

AI RESEARCH KNOWLEDGE HUB

Installation Guide

A repository-verified guide to installing, configuring, running, and validating the application.

Audience: Developers and system administrators

Basis: Current project source and configuration

Unverified deployment values are identified explicitly.

Contents

1.1 Purpose	4
1.2 System Overview	4
1.3 Prerequisites	4
1.4 Required Software	4
1.5 Hardware / Minimum System Requirements	5
1.6 Project Structure	5
1.7 Backend Installation	5
Local Windows setup	5
Docker setup	5
1.8 Frontend Installation	5
1.9 Database Setup	6
Docker Compose database	6
Local PostgreSQL database	6
1.10 Vector Database / Vector Store Setup	6
1.11 AI / LLM Configuration	6
1.12 Environment Variables	7
1.13 Database Migration / Initialization	8
1.14 Running the Backend	8
Local Windows	8
Docker Compose	8
1.15 Running the Frontend	9
1.16 Initial User / Admin Setup	9
1.17 Verification and Smoke Testing	9
1.18 Common Installation Problems	10
1.19 Troubleshooting	10
Database connection failure	10

Missing schema or lookup data 10

Ollama model unavailable 11

ChromaDB or embedding initialization failure 11

Frontend cannot reach the backend 11

Document processing failure 11

1.20 Production Deployment Notes **11**

Audience: Developers and system administrators

Verified against: Current repository configuration and implementation

Document status: Repository-verified; deployment-specific values are identified explicitly

Values that cannot be determined from this repository are stated as **To be confirmed from the deployment environment**. No production credentials are included in this guide.

1.1 Purpose

This guide explains how to install, configure, start, and verify the AI Research Knowledge Hub using only settings and behavior verified in this repository. It documents both the repository's Docker Compose setup and a local Windows development setup.

1.2 System Overview

The application consists of a React/Vite single-page frontend and a FastAPI backend. The backend uses SQLAlchemy with PostgreSQL for users, document metadata, extracted chunks, and audit-related records. It processes PDF and DOCX uploads, stores document files locally, creates text embeddings with SentenceTransformers, persists vectors in an embedded ChromaDB database, and calls a local Ollama service for RAG/chat and document summaries.

Authentication uses bearer JWTs signed with HS256; access tokens expire after 24 hours. Passwords are hashed with bcrypt. Roles defined by the application are `admin`, `researcher`, and `viewer`. Login events and other selected user activities are written to the audit log.

The system does not configure a separate ChromaDB server, hosted AI provider, OCR service, or database extension.

1.3 Prerequisites

Choose one installation path:

Path	Host prerequisites	Services that must be available
Docker Compose	Docker Desktop on Windows, or Docker Engine and Docker Compose V2 on Linux	Compose starts PostgreSQL, Ollama, backend, and frontend services. Ollama models must still be pulled after startup.
Local development	Python 3.11, Node.js (the repository's Docker images use Node 22), npm, and PowerShell on Windows	PostgreSQL and Ollama must already be running and reachable at the configured URLs.

For local development, provision a PostgreSQL database and database role that match `DATABASE_URL`. This repository does not provide a PostgreSQL installer or a script that creates the database role/database. The Compose database name is `knowledge_hub`; the local default connection is shown in `.env.example` and `backend/app/db/database.py`.

1.4 Required Software

Software	Repository evidence / role
Python 3.11	<code>backend/Dockerfile</code> uses <code>python:3.11-slim</code> ; the project README also specifies Python 3.11.
Node.js 22 and npm	Both frontend Dockerfiles/configurations use Node 22; <code>frontend/package-lock.json</code> supports reproducible <code>npm ci</code> installs.
PostgreSQL 16	<code>docker-compose.yml</code> and <code>docker-compose.dev.yml</code> use <code>postgres:16-alpine</code> .
Ollama	The backend calls the Ollama HTTP API; Compose uses the <code>ollama/ollama</code> image.
Docker Compose V2	Runs the multi-service stack declared in the Compose files.

Backend Python packages are pinned or bounded in `backend/requirements.txt`. Frontend dependencies and commands are declared in `frontend/package.json` and locked by `frontend/package-lock.json`.

1.5 Hardware / Minimum System Requirements

The repository does not state minimum CPU, RAM, GPU, or free-disk figures. These values are **To be confirmed from the deployment environment**.

Plan storage for uploaded documents under `data/documents/`, the persistent ChromaDB directory `vectorstore/`, the Ollama model volume, and the SentenceTransformers model cache. Their required capacity depends on document volume and selected model files; a numeric estimate is **To be confirmed from the deployment environment**. No GPU is configured or required explicitly by the application code.

1.6 Project Structure

project-root/	
backend/	
app/	FastAPI application, routers, models, and services
requirements.txt	Python dependencies
init_db.py	Create schema and seed lookup data
create_admin.py	Legacy user-creation helper; contains development credentials
data/	
documents/	Uploaded source documents
processed/	Present in the repository; no active storage configuration found
uploads/	Present in the repository; no active storage configuration found
database/migrations/	Empty in the checked repository
frontend/	
src/	React application and API client
package.json	npm scripts and dependencies
Dockerfile	Production build served by Nginx
vectorstore/	Persistent ChromaDB data (created as needed)
.env.example	Sample variables; not a complete list of code-supported variables
docker-compose.yml	Production-style multi-service Compose configuration
docker-compose.dev.yml	Development Compose configuration

Uploaded documents are stored under `data/documents/`. ChromaDB uses `vectorstore/`. In the production Compose file, both directories are mounted from the project directory into the backend container.

1.7 Backend Installation

Local Windows setup

Run these commands from the project root. They create a Python 3.11 virtual environment and install the exact backend requirements:

```
py -3.11 -m venv .venv311
.\.venv311\Scripts\python.exe -m pip install -r backend\requirements.txt
```

Successful installation ends with pip reporting that the listed packages were installed or are already satisfied. The dependency list includes FastAPI, Uvicorn, SQLAlchemy, the PostgreSQL driver, document parsers, ChromaDB, SentenceTransformers, PyTorch, authentication packages, and pytest. No migration framework or libmagic wrapper is currently included because neither is used by the application code.

Docker setup

The backend image installs `backend/requirements.txt` and starts `uvicorn app.main:app` on port 8002. Use the Docker procedure in Section 1.14 rather than installing backend packages on the host.

1.8 Frontend Installation

Run from the `frontend/` directory. `npm ci` installs the lockfile-exact dependency tree:

```
cd frontend
npm ci
```

Successful output ends with npm reporting the dependency installation completed. The development server is started separately in Section 1.15. The Vite development server defaults to port 5173; the application API client defaults to

`http://localhost:8002/api`.

Configuration is in `frontend/package.json`, `frontend/package-lock.json`, and `frontend/vite.config.js`. The Vite configuration enables the React plugin and does not define a custom proxy.

1.9 Database Setup

The application is configured for PostgreSQL through SQLAlchemy and `psycopg2-binary`. The default database URL in `backend/app/db/database.py` targets `localhost:5432/knowledge_hub`; its source-defined development account values are intentionally not reproduced here. Do not use those values in a real deployment. The Compose files use a PostgreSQL 16 Alpine container and a database named `knowledge_hub`.

Docker Compose database

The database service is created automatically by Compose. In `docker-compose.yml`, PostgreSQL is not published to a host port; the backend connects over the Compose network at `db:5432`. The database data is persisted in the named volume `postgres_data`. The development Compose file also publishes the database on host port 5432 and uses the named volume `postgres_dev_data`.

Local PostgreSQL database

Start a separately provisioned PostgreSQL service and set `DATABASE_URL` to its connection string. The database role, password, database creation procedure, and any deployment-specific PostgreSQL settings are **To be confirmed from the deployment environment**. No PostgreSQL extensions are created or required by the repository's initialization code.

1.10 Vector Database / Vector Store Setup

The backend uses ChromaDB's embedded `PersistentClient`; it is not a separately started network service. `backend/app/services/vector_store.py` creates the `vectorstore/` directory when necessary and uses the collection `research_documents`, which is created when first accessed.

No ChromaDB port, server, credentials, or external vector-store configuration is required. The production Compose file persists the directory with `./vectorstore:/app/vectorstore`. Back up this directory together with the uploaded source files and PostgreSQL database if vector data must be retained.

1.11 AI / LLM Configuration

Capability	Model / service	How it is used
RAG and chat generation	Ollama model <code>deepseek-r1:1.5b</code> by default	<code>LLM_MODEL</code> in <code>backend/app/services/llm_service.py</code> ; Ollama HTTP endpoint <code>/api/generate</code> .
Document summaries	Ollama model <code>qwen3.5:0.8b</code> by default	<code>SUMMARY_LLM_MODEL</code> in <code>backend/app/services/summarization_service.py</code> ; Ollama HTTP endpoint <code>/api/generate</code> .
Text embeddings	SentenceTransformers model <code>all-MiniLM-L6-v2</code>	Hard-coded default in <code>backend/app/services/embedding_service.py</code> ; normalized embeddings are stored in ChromaDB.

For a local Ollama installation, the default host is `http://localhost:11434`. In Compose, the backend uses `http://ollama:11434`. The Compose stack does not pull the models automatically. Start the stack first, then pull both configured Ollama models from the project root:

```
docker compose exec ollama ollama pull deepseek-r1:1.5b
docker compose exec ollama ollama pull qwen3.5:0.8b
```

For host-installed Ollama, start its service from a host terminal with `ollama serve` if it is not already running, then pull the same two model tags with `ollama pull <model-tag>`. Keep Ollama reachable at `OLLAMA_HOST`. Model downloads require network access the first time unless the model is already present. SentenceTransformers loads `all-MiniLM-L6-v2` through its library default model cache; the exact cache path is not configured by this project and is **To be confirmed from the deployment environment**.

1.12 Environment Variables

The table combines variables read by the application and variables passed by Compose. `.env.example` is the starting point, but it does not list every variable recognized by the backend. Values below are formats or defaults, not production credentials.

Variable	Purpose	Required	Example / format	Where used
DATABASE_URL	SQLAlchemy PostgreSQL connection string	Yes for database access; code has a local default	<code>postgresql://<user>:<password>@<host>:5432/knowledge_hub</code>	<code>backend/app/db/database.py</code> ; Compose overrides it with the internal db service URL.
SECRET_KEY	Signs and verifies JWT access tokens	Required by <code>docker-compose.yml</code> ; replace the insecure code fallback for every real deployment	A unique, high-entropy secret supplied outside source control	<code>backend/app/core/security.py</code> ; passed into the backend by Compose.
OLLAMA_HOST	Ollama API base URL	Required for AI features; code has a local default	<code>http://localhost:11434</code> locally; Compose uses <code>http://ollama:11434</code>	<code>llm_service.py</code> , <code>summarization_service.py</code> , and Compose.
LLM_MODEL	Ollama model for RAG/chat	Optional; default <code>deepseek-r1:1.5b</code>	Ollama model tag	<code>backend/app/services/llm_service.py</code> ; Compose supports an override.
SUMMARY_LLM_MODEL	Ollama model for summaries	Optional; default <code>qwen3.5:0.8b</code>	Ollama model tag	<code>backend/app/services/summarization_service.py</code> ; Compose supports an override.
DEFAULT_ADMIN_USER_NAME	Username for the first automatically created admin	Optional; default <code>admin</code>	<code>admin</code>	<code>backend/app/core/security.py</code> ; read only when the default admin does not already exist.
DEFAULT_ADMIN_EMAIL	Email for the first automatically created admin	Optional; default <code>admin@example.com</code>	Valid email address	<code>backend/app/core/security.py</code> ; read only during first-admin creation.
DEFAULT_ADMIN_PASSWORD	Password for the first automatically created admin	Strong value required before first startup in a secured deployment; code contains an insecure development fallback	Supply a private, unique value outside this guide and source control	<code>backend/app/core/security.py</code> ; Compose files do not currently pass this variable to the backend. Configure it through an approved deployment override before first startup.
VITE_API_BASE_URL	Frontend API base URL	Optional; frontend defaults to <code>http://localhost:8002/api</code>	<code>http://localhost:8002/api</code>	<code>frontend/src/services/api.js</code> ; Vite build argument in <code>frontend/Dockerfile</code> and production Compose.
POSTGRES_DB	Initial database name for the PostgreSQL container	Required by the Compose PostgreSQL image	<code>knowledge_hub</code> in the current Compose configuration	<code>docker-compose.yml</code> and <code>docker-compose.dev.yml</code> ; values are declared directly in the files.
POSTGRES_USER	Initial PostgreSQL role for the container	Required by the Compose PostgreSQL image	<code>postgres</code> in the current Compose configuration	<code>docker-compose.yml</code> and <code>docker-compose.dev.yml</code> ; values are declared directly in the files.
POSTGRES_PASSWORD	Initial password for the PostgreSQL container role	Required by the Compose PostgreSQL image	Set privately; the checked-in Compose files contain a development value	<code>docker-compose.yml</code> and <code>docker-compose.dev.yml</code> ; coordinate any change with <code>DATABASE_URL</code> .

`DEFAULT_ADMIN_*` variables are read by native backend processes when present in the process environment or a discovered dotenv file. They are not declared in `.env.example`, and the provided Compose services do not forward them. Docker bootstrap configuration is therefore **To be confirmed from the deployment environment**.

The backend Dockerfile also sets `PYTHONDONTWRITEBYTECODE`, `PYTHONUNBUFFERED`, and `PIP_NO_CACHE_DIR` for container runtime/build behavior. These are Dockerfile implementation flags, not application configuration options.

Do not commit `.env` files or secrets. The repository's `.gitignore` is empty, so it does not currently protect local `.env` files; verify ignore rules before staging files.

1.13 Database Migration / Initialization

No versioned database migration configuration or revision scripts are present in the checked repository, and the `database/migrations/` directory is empty. No migration framework is configured; do not run versioned migration commands based on this repository state.

On backend startup, `backend/app/main.py` calls SQLAlchemy `Base.metadata.create_all()` and applies a small set of additive compatibility `ALTER TABLE` statements for older documents and `audit_logs` tables. It also creates the default admin if no admin username exists.

To create tables and seed the repository's categories, document types, and keywords, run the existing initializer from `backend/` after PostgreSQL is available:

```
..\venv311\Scripts\python.exe init_db.py
```

The expected final message is `Database initialization complete!`. The same table-creation behavior also runs when the FastAPI application starts; `init_db.py` additionally seeds lookup data. It does not seed research documents.

1.14 Running the Backend

Local Windows

First ensure PostgreSQL and Ollama are running. From `backend/`, start FastAPI with the repository's documented command:

```
..\venv311\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8002
```

Successful startup leaves Uvicorn running and listening at `http://127.0.0.1:8002`. FastAPI's Swagger UI is at `http://127.0.0.1:8002/docs`; ReDoc is at `/redoc`; OpenAPI JSON is at `/openapi.json`.

The health route is `GET /api/health`. It returns a JSON status response. For a real database check, also use an authenticated database-backed endpoint such as `GET /api/documents/` or `GET /api/dashboard/summary`; the health handler itself does not execute a database query.

Docker Compose

From the project root, make a local environment file from the sample, set a private `SECRET_KEY` in the current PowerShell session, and build/start the stack:

```
Copy-Item .env.example .env
$env:SECRET_KEY = "<generate-a-unique-secret-locally>"
docker compose up --build
```

Replace the placeholder locally; do not put a real secret in this guide or commit it. The production Compose file refuses to start if `SECRET_KEY` is empty. Successful startup shows the PostgreSQL, Ollama, backend, and frontend containers running. The backend API is exposed on host port 8002; frontend is exposed on 5173.

The `docker-compose.dev.yml` alternative starts Vite in development mode and publishes PostgreSQL, Ollama, backend, and frontend ports. Start it from the project root with:

```
docker compose -f docker-compose.dev.yml up --build
```

Its default JWT secret is explicitly development-only; do not use that configuration for production.

1.15 Running the Frontend

After installing dependencies, run from `frontend/`:

```
npm run dev
```

Vite prints the local URL, normally `http://localhost:5173/`. The application calls `http://localhost:8002/api` unless `VITE_API_BASE_URL` is supplied when Vite starts. The backend CORS configuration allows the local frontend origins on ports 5173 through 5176 and 3000, for localhost and 127.0.0.1.

To verify a production frontend bundle, run from `frontend/`:

```
npm run build
```

Vite writes the static output to `frontend/dist/`. The frontend Docker image serves that output with Nginx on container port 80; the Compose host mapping is 5173:80.

1.16 Initial User / Admin Setup

At backend startup, `ensure_default_admin()` creates an admin account if one does not already exist. Set `DEFAULT_ADMIN_PASSWORD` in the backend process environment before the first startup in local development. The code fallback is development-only; the credential is intentionally not reproduced in this guide. After signing in, change the password using the authenticated `POST /api/auth/change-password` endpoint. The API documents the request schema in Swagger.

Important: the provided Compose files do not forward `DEFAULT_ADMIN_PASSWORD`, `DEFAULT_ADMIN_USERNAME`, or `DEFAULT_ADMIN_EMAIL`. Secure first-admin bootstrap for Compose must be supplied through a deployment-approved override; exact deployment instructions are **To be confirmed from the deployment environment**. Do not assume the checked-in Compose defaults are production-safe.

Public registration, where enabled, creates a `viewer` account and does not grant administrative privileges. Admin and researcher roles are managed through authenticated administrative functionality. The `backend/create_admin.py` helper contains fixed development account values; do not use those values for a real deployment.

1.17 Verification and Smoke Testing

Use the following checklist after installation. AI-related checks require both Ollama models to be available and at least one processed document.

- [] **Backend starts:** Uvicorn remains running and its logs show it listening on port 8002.
- [] **API documentation opens:** visit `http://localhost:8002/docs` and confirm the FastAPI Swagger UI loads.
- [] **Health endpoint responds:** request `GET http://localhost:8002/api/health` and confirm the JSON response reports `healthy`.
- [] **Database access works:** authenticate and request `GET /api/documents/` or `GET /api/dashboard/summary`; the route should return a successful response rather than a database error.
- [] **Authentication works:** use `POST /api/auth/login` in Swagger with the deployment-configured admin account; confirm a bearer token is returned, then use **Authorize** and call `GET /api/auth/me`.
- [] **Backend tests pass:** from `backend/`, run `..\venv311\Scripts\python.exe -m pytest`; pytest should report the collected tests passing.
- [] **Frontend loads:** open the Vite or Compose frontend URL and sign in.
- [] **Document upload and processing work:** as an authorized researcher/admin, upload a text-searchable PDF or DOCX no larger than 50 MB; confirm processing status completes and chunks are stored.
- [] **Metadata is stored:** confirm the document title, author, publication year, and other available metadata appear in document details.
- [] **Semantic search works:** search for a phrase present in the uploaded document and inspect returned excerpts and source metadata.

- [] **Ask AI works:** ask a question grounded in the uploaded document; confirm an answer and source citations are returned.
- [] **Summarization works:** request a summary for a processed document and confirm a structured summary is returned.
- [] **Audit logging works:** as an administrator, open the activity log after login, search, upload, chat, or summary actions.

Some endpoints require authentication. Use the bearer token returned by `/api/auth/login` in Swagger's **Authorize** control. GET `/api/health` alone does not prove database connectivity because its handler returns a static status object.

1.18 Common Installation Problems

Problem	Likely cause	Resolution
Compose refuses to start because <code>SECRET_KEY</code> is missing	The production Compose file requires this variable	Set a unique local <code>SECRET_KEY</code> before <code>docker compose up --build</code> ; do not commit it.
Backend cannot connect to PostgreSQL	Wrong <code>DATABASE_URL</code> , database not provisioned, or database service not ready	Confirm the URL uses the correct host for the run mode: <code>localhost</code> from the host, <code>db</code> from Compose. Confirm the database and role exist.
Ask AI or summaries fail while the rest of the app works	Ollama is unreachable or the corresponding model is not present	Verify <code>OLLAMA_HOST</code> , start Ollama, and pull the exact configured model tags.
First semantic indexing is slow or fails	The embedding model is being downloaded, model-cache access is blocked, or model initialization failed	Allow model download access and writable cache storage; inspect backend logs.
Frontend loads but API requests fail	Backend is stopped, <code>VITE_API_BASE_URL</code> points to the wrong URL, or browser origin is outside configured CORS origins	Confirm port 8002, the frontend API URL, and the allowed origins in <code>backend/app/main.py</code> . Rebuild the frontend container after changing its Vite build argument.
Upload is rejected	Unsupported extension, mismatched content type, or file exceeds the limit	Use PDF or DOCX with matching MIME type and a size no greater than 50 MB.
PDF upload completes with little/no extracted text	The PDF may be image-only; the repository extracts PDF text with <code>pypdf/pdfplumber</code> and does not configure OCR	Use a text-searchable PDF. An OCR installation/workflow is To be confirmed from the deployment environment .
Uploaded files or vectors disappear after container replacement	Persistent mounts/volumes were removed or a different Compose project path was used	Preserve <code>data/</code> , <code>vectorstore/</code> , <code>postgres_data</code> , and <code>ollama_data</code> as applicable; do not remove volumes when bringing the stack down.

1.19 Troubleshooting

Database connection failure

Problem: startup or database-backed API calls fail.

Likely cause: `DATABASE_URL` does not match the active database host, port, database name, or provisioned account.

Solution: check the URL without sharing its password. Use `localhost` for a host-run backend and the Compose service name `db` for a containerized backend. Confirm PostgreSQL is accepting connections and that the `knowledge_hub` database exists. Database user provisioning is **To be confirmed from the deployment environment**.

Missing schema or lookup data

Problem: tables or category/type/keyword lookup data are absent.

Likely cause: initialization was not run against the intended database.

Solution: from `backend/`, run `..\.venv311\Scripts\python.exe init_db.py`. Backend startup also creates tables, but lookup seeding is performed by this script.

Ollama model unavailable

Problem: model checks warn that a model is unavailable or generation returns a connection/model error.

Likely cause: Ollama is stopped, `OLLAMA_HOST` is wrong, or the model has not been pulled.

Solution: confirm the Ollama service is reachable and that the configured `LLM_MODEL` and `SUMMARY_LLM_MODEL` tags exist in Ollama. In Docker, use `docker compose exec ollama ollama pull <model-tag>`.

ChromaDB or embedding initialization failure

Problem: vector indexing/search logs show model initialization or file access errors.

Likely cause: SentenceTransformers cannot download/load `all-MiniLM-L6-v2`, or the backend cannot write `vectorstore/`.

Solution: allow outbound model download access on first use, ensure the model cache and `vectorstore/` are writable, and check backend logs. The exact model-cache directory is **To be confirmed from the deployment environment**.

Frontend cannot reach the backend

Problem: login or API requests fail in the browser although the frontend renders.

Likely cause: wrong `VITE_API_BASE_URL`, backend not listening on `8002`, or CORS origin mismatch.

Solution: inspect the browser network request URL, verify `http://localhost:8002/api`, and check the origin allow-list in `backend/app/main.py`. For the Docker production frontend, change the Vite build argument and rebuild the image; the URL is compiled into the static bundle.

Document processing failure

Problem: a document remains failed or has no chunks.

Likely cause: unsupported file type, unreadable/corrupt PDF or DOCX, oversized upload, filesystem permissions, or unavailable embedding model.

Solution: check backend logs and `processing_error`, use a valid PDF/DOCX under 50 MB, confirm `data/documents/` is writable, then retry. OCR for scanned PDFs is not configured in this project.

1.20 Production Deployment Notes

- Set a unique `SECRET_KEY`; the backend source has an insecure fallback and the development Compose file supplies a development-only fallback.
- Configure first-admin credentials before startup. The checked-in Compose files do not pass `DEFAULT_ADMIN_*` variables to the backend; a secure production override is **To be confirmed from the deployment environment**.
- The production Compose file currently embeds PostgreSQL account values in the service configuration and the backend connection URL. Replace them through deployment-managed configuration before production; actual credentials and secret-management procedure are **To be confirmed from the deployment environment**.
- The production Compose file publishes the backend on `8002`, Ollama on `11434`, and frontend on `5173`. Keep internal services private unless access is explicitly needed. The PostgreSQL service is not host-published in that file.
- The frontend Docker image serves static files with Nginx; no TLS termination, reverse proxy, domain, or production CORS configuration is included. Those values are **To be confirmed from the deployment environment**.
- Preserve and back up the PostgreSQL volume, `data/` uploads, `vectorstore/`, and Ollama model volume. The repository does not include a production backup/restore procedure; this is **To be confirmed from the deployment environment**.
- The Compose Ollama image uses the mutable `latest` tag. A production image pinning policy is not defined in the repository and is **To be confirmed from the deployment environment**.
- Registration creates viewer-role accounts; restrict administrative access and assign elevated roles only through the existing administrative functionality.

- Protect uploaded research files and do not expose PostgreSQL, Ollama, or backend ports beyond the required network boundary. Production firewall rules and retention policy are **To be confirmed from the deployment environment**.
- Never commit `.env` files, model files, database dumps, uploaded documents, or secret values. The current `.gitignore` is empty; add and verify appropriate repository ignore rules as part of deployment preparation.

Configuration sources reviewed: `.env.example`, `README.md`, `docker-compose.yml`, `docker-compose.dev.yml`, `backend/requirements.txt`, backend Dockerfile and FastAPI/database/security/service modules, frontend `package.json`, Vite and Docker configuration, and the repository's initialization scripts.